

Tarea 5

Pruebas automatizadas

Integración, regresión y proceso de CI

Organízate - Cuidado del adulto mayor
CC3091 - Ingeniería de Software 2
Semestre II, 2026

Integrante	Carné
Luis Lee	241203
Belen Monterroso	231497
Sebastian Lemus	241155
Luis Hernandez	241424

Fecha de verificación local: 6 de octubre de 2026
Zona horaria: America/Mexico_City

Entrega: investigación comparativa, seis pruebas específicas, automatización, reportes originales, experimento reproducible y conclusiones. La grabación del video de 5 a 8 minutos queda pendiente; el guion está preparado.

1. Objetivo y estrategia

La estrategia comprueba interacciones reales entre la API de Organízate, sus validadores, servicios, modelos y persistencia. Además, protege tres comportamientos implementados: impedir horarios invertidos, conservar el rol al actualizar el perfil y bloquear el inicio de sesión de cuidadores pendientes de aprobación.

Contexto técnico

El proyecto utiliza Laravel 11, PHP 8.2, Sanctum y Eloquent; el frontend utiliza HTML, CSS y JavaScript con módulos ES. La base de datos del entorno de aplicación es PostgreSQL. Para pruebas se emplea SQLite en memoria, configurada en backend/phpunit.xml. Los paquetes locales verificados son PHPUnit 11.5.55 y Vitest 3.2.7, con jsdom y Testing Library.

Capas y criterios de aceptación

Capa	Verificación y alcance
Integración	I01-I03: peticiones HTTP de prueba ejecutadas en Laravel, servicios y modelos reales, migraciones y consultas a SQLite. No se reemplaza la base de datos ni la lógica de negocio con mocks.
Regresión	R01-R03: reglas conocidas que deben conservarse. Aserciones sobre respuesta HTTP y estado persistido; mismos datos y expectativas antes y después del cambio.
Frontend complementario	Vitest prueba utilidades y comportamiento DOM. Las respuestas API simuladas de estas pruebas no se presentan como integración frontend-API real.
Automatización	Proceso Node que ejecuta PHP en procesos separados, conserva JUnit/logs y valida el ciclo aprobado-fallo-aprobado. Workflow por push y pull_request.

Cada caso inicia con datos sintéticos y RefreshDatabase. I01 usa inicio de sesión y un token Sanctum real. En I02, I03, R01 y R02 se fija la identidad con Sanctum::actingAs para concentrar el caso en su regla; validadores, controladores, servicios y persistencia continúan siendo reales. R03 prueba credenciales contra la autenticación real.

Reproducibilidad y límites de interpretación

Las peticiones HTTP se procesan dentro del kernel de Laravel; no cruzan una conexión de red. SQLite es persistencia real durante la prueba, pero no prueba particularidades de PostgreSQL. Los seis casos nuevos se suman a la suite existente; el total del backend incluye pruebas de distintos tipos y no se clasifica íntegramente como integración.

2. Comparación de frameworks

Se compararon tres alternativas compatibles con las tecnologías del proyecto. La selección es una valoración técnica del equipo basada en la documentación oficial [1]-[5] y la ejecución local.

Criterio	PHPUnit + Laravel [1,2]	Vitest + jsdom [3]	Playwright [4,5]
Compatibilidad	PHP/Laravel y Eloquent; instalado.	JS y módulos ES del frontend; instalado.	Frontend web en navegador; también permite pruebas API.
Configuración	Composer, phpunit.xml, factories y RefreshDatabase.	npm, vitest.config.js, entorno jsdom y setup.	Paquete npm, navegadores, servidor y datos de prueba.
Tipos de pruebas	Unitarias y Feature; integración y regresión HTTP/persistencia.	Unitarias, DOM y regresión; API real requiere preparación adicional.	Funcionales E2E, API y regresión visual.
Flujo de desarrollo	CLI PHP; fácil ejecución en push y PR.	CLI/watch; ejecución con npm en CI.	CLI; servidor y navegadores en CI.
Reportes	JUnit XML y salida TestDox.	Consola y JUnit XML.	HTML, JUnit, trazas y capturas.
Documentación	Manual versionado y guía HTTP de Laravel.	Guía oficial, configuración y reporters.	Guías oficiales de pruebas, CI y reportes.
Ventajas	Aserciones HTTP y de BD; reutiliza el entorno del backend.	Aprovecha las 49 suites del frontend con ejecución sencilla.	Comprueba interacción en navegadores reales.
Limitaciones	No demuestra red ni interfaz de navegador; BD elegida afecta alcance.	jsdom y API simulada no prueban integración real con servidor.	Mayor preparación y dependencia de servidor, datos y selectores.

Selección y justificación

PHPUnit con Laravel es la herramienta principal: sus casos ejercitan rutas, servicios y SQLite sin introducir otro framework PHP. **Vitest** complementa con la suite JS existente. **Playwright** se reserva para ampliar E2E: aporta navegador real, pero no es necesario para las integraciones API-persistencia elegidas. La decisión responde a compatibilidad, evidencia y costo de preparación, no al uso de una herramienta en clase.

3. Comparación de integración continua

Las tres alternativas pueden ejecutar Composer, PHP, Node y pruebas. Los detalles siguientes proceden de las guías oficiales [6]-[8]; la preferencia por Actions se fundamenta también en que el remoto del proyecto está alojado en GitHub.

Criterio	GitHub Actions [6]	GitLab CI/CD [7]	Jenkins [8]
Compatibilidad	Runners para comandos PHP/Node; conexión directa con el repositorio GitHub.	Runners e imágenes capaces de ejecutar PHP/Node.	Agentes configurables con PHP/Node.
Configuración	YAML en .github/workflows y pasos de preparación.	.gitlab-ci.yml y runner; requiere integración con GitLab.	Jenkinsfile, servidor Jenkins, agentes y plugins necesarios.
Pruebas soportadas	Ejecuta cualquier suite CLI; no define las aserciones.	Ejecuta suites CLI en jobs y stages.	Ejecuta suites CLI mediante etapas del pipeline.
Flujo de desarrollo	Eventos push, pull_request y disparo manual.	Pipelines por commits, merge requests y ejecución manual.	Pipeline conectado al SCM y disparadores configurables.
Resultados	Estado de jobs, logs, resumen y archivos como artefactos.	Estado de pipelines/jobs y artefactos; reportes de pruebas configurables.	Historial y logs; publicación JUnit con plugins.
Documentación	Guías oficiales de eventos, runners y workflows.	Guías de YAML, runners y pipelines.	Manual de Pipeline y Jenkinsfile.
Ventajas	Menor preparación para este remoto; evidencia junto al código.	Configuración declarativa y control mediante runners.	Control de infraestructura y amplia personalización.
Limitaciones	Depende de runners, permisos y disponibilidad de GitHub.	Añade configuración o migración para el flujo actual en GitHub.	El equipo debe mantener servidor, agentes y plugins.

Combinación elegida

PHPUnit + Laravel, Vitest y GitHub Actions. Actions agrega automatización al repositorio existente, prepara ambos runtimes y conserva archivos de evidencia. GitLab CI sería razonable si el equipo trasladara su flujo a GitLab. Jenkins sería útil con infraestructura propia y necesidades de control que aquí no justifican su mantenimiento adicional.

4. Pruebas de integración implementadas

Código nuevo: backend/tests/Feature/Tarea5/IntegrationTest.php. Todos los casos usan RefreshDatabase y comprueban una respuesta de la API y datos persistidos, sin mock de servicios ni base de datos.

I01. Login, token y perfil

Componentes: ruta POST /api/login, AuthController/AuthService, hash, User, tabla personal_access_tokens, Sanctum y GET /api/me.

Condición: el token emitido para credenciales válidas existe en la base de datos y permite leer el perfil correcto.

Datos: profesional aprobado, integracion@example.com y contraseña secret123 almacenada mediante hash.

Resultado: login 200, token de tipo string, fila vinculada al usuario y perfil 200 con el mismo id. Cinco aserciones. La autenticación del perfil usa el token emitido, no actingAs.

I02. Adulto mayor, medicamento e ingreso de inventario

Componentes: POST y GET /api/admin/older-adults, OlderAdultRequest, OlderAdultController/OlderAdultService, Eloquent y tablas older_adults, medications, older_adult_medications y medication_acquisitions.

Condición: crear un adulto con medicación conserva sus relaciones y el ingreso inicial de inventario; una lectura posterior devuelve el adulto creado.

Datos: administrador, familiar y profesional aprobados; Rosa Integracion, 82 años, habitación T5-101; Losartan T5, 1 tableta a las 08:00, lunes, cantidad 7, mínimo 2.

Resultado: creación 201; medicamento en JSON; filas del adulto, medicamento, asignación y adquisición con cantidad 7; consulta posterior 200. Ocho aserciones.

I03. Nota de seguimiento y relaciones

Componentes: POST /api/professional/routine-notes, validación, ProfessionalRoutineNoteController, RoutineNoteService, comprobación de asignación, modelos y tabla routine_notes.

Condición: la nota creada vincula al profesional autenticado y al adulto asignado.

Datos: profesional aprobado; Rosa Nota, habitación T5-102, asignada a ese profesional; contenido Seguimiento de integracion T5.

Resultado: HTTP 201, ids correctos en JSON y fila con el contenido y ambas claves en la base de datos. Cuatro aserciones.

Qué aporta la integración real

Un servicio que omita guardar la adquisición, una relación con id incorrecto, un contrato JSON modificado o un token que no se persista puede hacer fallar estos casos, aunque los métodos aislados continúen funcionando. La prueba atraviesa más de un componente real y verifica el efecto observable en la persistencia.

5. Pruebas de regresión implementadas

Código nuevo: backend/tests/Feature/Tarea5/RegressionTest.php. Las funcionalidades ya existían y estaban cubiertas por AuthenticationTest, ProfileRoleProtectionTest y CaregiverScheduleEndTimeValidationTest. Los casos específicos fijan una línea base de la tarea y protegen el mismo comportamiento en ejecuciones posteriores.

R01. Rechazo de horarios invertidos

Comportamiento protegido: end_time debe ser posterior a start_time y un horario inválido no debe persistirse.

Componentes: POST /api/schedules, SaveCaregiverScheduleRequest, controlador, CaregiverScheduleService y tabla caregiver_schedules.

Datos: profesional aprobado, día 3, inicio 16:00 y fin 08:00.

Resultado esperado: 422, error de validación de end_time y cero horarios. Cuatro aserciones.

Cambio que provoca regresión: retirar after:start_time del validador. Este es el cambio deliberado demostrado experimentalmente.

R02. Protección del rol al actualizar perfil

Comportamiento protegido: un profesional puede actualizar su nombre, pero no convertirse en administrador mediante PUT /api/me.

Componentes: AuthRequest, AuthController/AuthService, User y tabla users.

Datos: profesional aprobado; nombre Profesional T5; email actual; campo inyectado role=admin.

Resultado esperado: 200 para la modificación permitida; JSON y base de datos mantienen role=profesional y guardan el nombre. Tres aserciones.

Cambio que podría provocar regresión: aceptar role en los campos de actualización o pasar todo el payload a User::update.

R03. Bloqueo de cuidadores pendientes

Comportamiento protegido: tener credenciales correctas no basta para iniciar sesión si el cuidador aún no está aprobado.

Componentes: POST /api/login, AuthService, User, hash y tokens Sanctum.

Datos: profesional con is_approved=false y contraseña secret123 válida.

Resultado esperado: HTTP 403 y cero filas en personal_access_tokens. Dos aserciones.

Cambio que podría provocar regresión: eliminar la comprobación de aprobación, invertir la condición o crear un token antes de rechazar al usuario.

Criterio de detección

Una respuesta correcta sin comprobar datos podría ocultar una escritura indebida. Por ello los casos también verifican ausencia de horarios y tokens o preservación del rol. Se mantienen las expectativas originales en las tres fases; no se altera la prueba para lograr un resultado verde.

6. Experimento: aprobado, fallo y recuperación

Ejecución local real del 6 de octubre de 2026, registrada por tests/automation/run.mjs. Cada fase inicia un proceso PHP separado y usa los mismos seis casos. Los reportes JUnit y registros originales se conservan en tests/automation/reports/.

Fase	Pruebas	Aserciones	Fallos	Salida	Tiempo total
baseline	6	26	0	0	3.106 s
regression	6	23	1	1	2.757 s
corrected	6	26	0	0	2.425 s

Cambio deliberado en el componente real

Original:

```
'end_time' => ['required', 'date_format:H:i', 'after:start_time'],
```

Mutacion:

```
'end_time' => ['required', 'date_format:H:i'],
```

El script copia la clase SaveCaregiverScheduleRequest y retira únicamente la regla indicada. PHP carga esa copia mediante auto_prepend_file antes de que Laravel resuelva el validador. Se ejecuta así la API con el componente modificado, incluyendo su escritura real en SQLite. El código original no se sobrescribe.

Evidencia de la detección

```
...F.. 6 / 6
```

```
Expected response status code [422] but received 201.
```

```
Tests: 6, Assertions: 23, Failures: 1.
```

R01 falla porque el sistema crea el horario invertido; los otros cinco casos aprueban. El runner exige código 1, exactamente un fallo de aserción, cero errores y el mensaje 201 frente a 422. Un fallo de infraestructura no se aceptaría como evidencia de regresión.

Corrección y comprobación

La fase corrected deja de cargar la copia modificada y recupera la regla original after:start_time. Las seis pruebas vuelven a aprobar con 26 aserciones. El manifiesto registra source_restored=true y experiment_success=true; los hashes SHA-256 del archivo original antes y después coinciden.

```
node tests/automation/run.mjs baseline
node tests/automation/run.mjs regression # salida 1 esperada
node tests/automation/run.mjs corrected
node tests/automation/run.mjs demo      # valida todo el ciclo
```

7. Proceso de integración continua

Configuración: `.github/workflows/tarea5-tests.yml`. Eventos: `push`, `pull_request` y `workflow_dispatch`. Dos jobs en Ubuntu preparan PHP 8.2/Node 22 y ejecutan los comandos del repositorio. El workflow tiene permiso `contents:read` y no despliega la aplicación.

Etapa	Acción y evidencia
Preparación backend	Checkout; PHP con mbstring, PDO SQLite, DOM y XML; Node; composer install desde <code>composer.lock</code> .
Integración/regresión	<code>node tests/automation/run.mjs demo</code> : verifica línea base, detecta el fallo deliberado y verifica la recuperación. Un fallo inesperado devuelve código no cero.
Suite backend	<code>node tests/automation/run.mjs full</code> después del experimento exitoso: revisa también la suite existente.
Frontend	<code>npm ci</code> desde <code>package-lock.json</code> ; <code>npm test</code> con reporters de consola y JUnit.
Conservación	<code>upload-artifact</code> con <code>if:always()</code> ; JUnit, logs, manifiestos y diff del backend, más reporte frontend. Retención: 30 días.
Resultado visible	Estado verde/rojo del job, consola y resumen de ejecución. La fase manual regression produce un job rojo deliberadamente.

Los reportes del CI se generan en `ci-reports/`, separado de la evidencia local versionada, para evitar mezclar resultados históricos con una ejecución nueva. Las pruebas utilizan la base SQLite en memoria; las migraciones se preparan mediante `RefreshDatabase`.

Interpretación del experimento en CI

En demo, la fase regression falla por diseño y el orquestador valida esa detección antes de continuar. El job completo termina verde solo si aprueban `baseline`, la comprobación del fallo esperado, `corrected` y la suite completa. En `workflow_dispatch`, elegir regression por separado hace que su código 1 se propague y el job aparezca rojo. Los artefactos se guardan en ambos casos.

Estado de la evidencia remota

El workflow está implementado y los mismos comandos fueron verificados localmente. Al generar esta versión todavía no se ha conservado una ejecución remota de GitHub Actions. La evidencia local no se presenta como ejecución del servicio remoto. La publicación de la rama activará el evento `push`.

8. Resultados, esfuerzo y configuración

Ejecución local	Pruebas	Resultado	Tiempo
Casos específicos Tarea 5	6	0 fallos; 26 aserciones	3.106 s (proceso)
Mutación controlada	6	1 fallo R01; 5 aprobadas	2.757 s (proceso)
Recuperación	6	0 fallos; 26 aserciones	2.425 s (proceso)
Backend completo	186	0 fallos; 1660 aserciones	64.503 s (proceso)
Frontend completo	4	49 archivos aprobados; 0 fallos	35.34 s (Vitest)

El backend completo incluye los seis casos nuevos; no deben sumarse nuevamente a su total. Backend y frontend ofrecen 309 pruebas distintas aprobadas en la verificación amplia. Los tiempos del runner incluyen arranque PHP y escritura de reportes; el tiempo Vitest es el informado por su consola. Son mediciones de esta máquina, no un SLA ni una medición de GitHub.

Tiempo y esfuerzo

El experimento de tres fases consumió aproximadamente 8.29 segundos de procesos medidos. La suite completa del backend requirió 64.50 segundos y Vitest 35.34 segundos; estos últimos procesos se ejecutaron de forma concurrente. El esfuerzo principal consistió en elegir fronteras reales, preparar datos y aserciones, aislar la mutación, manejar códigos de salida y producir evidencia legible. No se dispone de un registro de horas-persona del equipo, por lo que no se declara una cifra ficticia.

Problemas encontrados y respuesta

Ruta del ejecutable: la primera invocación de PHPUnit omitía el último segmento; se utilizó `vendor/phpunit/phpunit/phpunit` con `cwd` explícito desde el runner.

Estado cambiante del proyecto: una ejecución exploratoria amplia coincidió con cambios de configuración del backend y mostró errores de hash y preflight CORS. Se repitió la suite sobre el estado actualizado y terminó con 186 pruebas y cero fallos; no se atribuyen esas correcciones al código de esta tarea.

Falso positivo de mutación: se añadió una comprobación estricta del fallo esperado y de cero errores para rechazar crashes.

Reportes históricos: se separó el directorio de CI del directorio de evidencia local.

Utilidad observada

PHPUnit dio una explicación precisa del contrato roto y comprobó persistencia. Vitest validó la suite JS existente. El runner produjo registros y manifiestos reproducibles, y Actions integra su ejecución con cambios del repositorio. El fallo deliberado evidencia sensibilidad a una regresión concreta; no prueba que toda regresión posible esté cubierta.

9. Alcance y conclusiones

Puede detectar	Fuera del alcance actual
Cambio de estados HTTP o claves JSON en los flujos probados.	Flujos y combinaciones de datos sin casos automatizados.
Ausencia de tokens, notas, medicamentos, relaciones o adquisiciones esperadas.	Problemas específicos de PostgreSQL, concurrencia o transacciones bajo carga.
Pérdida de validación temporal, aprobación o protección del rol.	Auditoría exhaustiva de seguridad, ataques nuevos y toda configuración productiva.
Regresiones en utilidades y DOM cubiertas por Vitest.	Renderizado real, CSS, accesibilidad completa y compatibilidad de navegadores.
Fallos de instalación y ejecución en el pipeline.	Disponibilidad de servicios externos, latencia de red y desempeño productivo.

Conclusiones sustentadas

1. Las tres integraciones aprobadas comprueban efectos entre API y persistencia: token-perfil, adulto-medicación-inventario y nota-asignación. Su evidencia va más allá de verificar un método aislado.
2. Los tres casos de regresión fijan comportamientos con consecuencias operativas y de acceso. El experimento de horarios demuestra que R01 detecta la eliminación de la regla y que la corrección recupera el comportamiento.
3. Se mantendrían PHPUnit/Laravel y Vitest: están instalados, ofrecen reportes utilizables y aprobaron sus suites. Se mantendría Actions por su ajuste al repositorio GitHub y automatización declarativa; su estado remoto debe verificarse en la evidencia del servicio.
4. SQLite permite aislamiento y rapidez, pero no sustituye validar PostgreSQL. Una siguiente ampliación útil es un job contra PostgreSQL y pruebas Playwright de flujos críticos en navegador.
5. El experimento controlado proporciona una evidencia concreta de detección. La confianza depende de mantener datos, contratos y escenarios representativos al evolucionar el sistema.

Material de entrega y grabación

El PDF documenta investigación, elección, casos, evidencia y conclusiones. El repositorio contiene las pruebas, el runner, el workflow y los reportes. docs/TAREA5_GUION_VIDEO.md prepara una demostración de aproximadamente 7 minutos con responsables sugeridos, comandos, resultados y explicación del CI. La grabación y su enlace se agregarán al realizar el video.

10. Trazabilidad y referencias

Requisito / evaluación	Evidencia
Investigación de frameworks y CI (15 puntos)	Secciones 2 y 3: tres alternativas de cada categoría y criterios mínimos.
Justificación (15 puntos)	Selección en secciones 2 y 3; contexto del proyecto en sección 1.
Diseño pertinente (20 puntos)	I01-I03 y R01-R03: condición, componentes, datos, resultado y riesgos de cambio.
Implementación (20 puntos)	Dos clases nuevas de Tarea5; JUnit y logs de baseline/corrected.
Proceso automatizado (15 puntos)	Workflow push/PR, instalación, dos jobs y artefactos; sección 7.
Detección y recuperación (10 puntos)	Regla retirada, R01 en fallo, recuperación y manifiesto; sección 6.
Análisis y conclusiones (5 puntos)	Resultados, esfuerzo, problemas y límites; secciones 8 y 9.
Video de 5 a 8 minutos	Guion preparado; grabación pendiente.

Fuentes primarias consultadas el 6 de octubre de 2026. Las comparaciones se parafrasean; no se copian artículos ni se atribuyen resultados de ejecución a la documentación.

[1] Laravel 11 - HTTP Tests. <https://laravel.com/docs/11.x/http-tests>

[2] PHPUnit 11.5 - Command-Line Test Runner. <https://docs.phpunit.de/en/11.5/textui.html>

[3] Vitest - Reporters. <https://vitest.dev/guide/reporters>

[4] Playwright - Continuous Integration. <https://playwright.dev/docs/ci>

[5] Playwright - Test Reporters. <https://playwright.dev/docs/test-reporters>

[6] GitHub - Understanding GitHub Actions. <https://docs.github.com/en/actions/get-started/understand-github-actions>

[7] GitLab - Get started with CI/CD. <https://docs.gitlab.com/ci/>

[8] Jenkins - Pipeline. <https://www.jenkins.io/doc/book/pipeline/>

Fuente de requisitos: Tarea 5. Pruebas automatizadas. Integración, Regresión, Funcionales. 2026.pdf, Universidad del Valle de Guatemala, CC3091, 2 páginas. Fuente experimental: archivos del proyecto y reportes generados por PHPUnit/Vitest incluidos con esta entrega.